정보보안학 기초 Part01

챕터별 상세 요약 정리

암호의 탄생부터 악성코드 대응까지 · 10개 챕터 종합 정리

목차

1장. 암호의 탄생과 진화 이야기

2장. 대칭키 암호화의 원리와 활용

3장. 공개키 암호화의 혁신적 발견

4장. 해시함수와 디지털 서명의 비밀

5장. 키 관리와 인증서의 세계

6장. 네트워크 보안의 기초와 실제

7장. 웹 보안과 사이버 공격들

8장. 무선 네트워크 보안의 이해

9장. 운영체제 보안과 시스템 보호

10장. 데이터베이스 보안과 악성코드의 진화

1 장. 암호의 탄생과 진화 이야기

인류가 비밀을 지키려는 욕구는 문명과 함께 시작되었으며, 암호는 처음부터 권력과 지식을 보호하는 수단으로 사용되어 수천 년에 걸쳐 발전해 왔습니다.

고대의 암호

- **이집트 상형문자 암호** — 기원전 1900 년경 귀족 크누마텝 2 세의 무덤 벽화에 일반 상형문자를 다른 문자로 대체한 기록이 남아 있으며, 이는 최초로 기록된 암호 사용 사례 중 하나입니다. 신성한 의식과 비밀 지식을 일반인으로부터 숨기려는 목적이었습니다.
- **카이사르 암호** — 로마의 율리우스 카이사르가 사용한 방법으로, 알파벳을 일정 칸(예: 3 칸) 뒤로 밀어 치환합니다(A→D, B→E). '몇 칸 이동'이 바로 '키'가 되며, 규칙적 치환을 사용한 최초의 암호 중 하나입니다.

빈도 분석 — 단순 암호의 약점

카이사르 암호는 단일 알파벳 치환 방식이어서 치명적 약점이 있었습니다. 9세기 아랍 수학자 알킨디가 개발한 빈도 분석이 그것으로, 언어마다 특정 글자가 자주 등장하는 통계적 특성(한국어의 'ㅇ', 영어의 E·T)을 이용해 암호문에서 가장 자주 나오는 글자를 원래 글자로 추론합니다. 이로써 단순 치환 암호는 비교적 쉽게 해독됩니다.

다중 알파벳 암호

- **알베르티의 암호판** — 15 세기 이탈리아의 레온 바티스타 알베르티가 여러 개의 다른 암호 알파벳을 사용하는 방법을 고안했습니다.
- **비제네르 암호** — 16 세기 프랑스의 블레즈 드 비제네르가 발전시킨 다중 알파벳 치환 암호로, 키워드를 반복 사용해 같은 글자라도 위치에 따라 다른 글자로 변환되게 만들어 빈도 분석을 무력화했습니다.

20 세기 — 암호학의 혁명

1·2차 대전을 거치며 암호는 전쟁의 승패를 좌우하는 핵심 도구가 되었습니다.

- **에니그마** — 독일군이 개발한 기계식 회전자 암호 장치. 키를 누를 때마다 원판이 회전해 같은 글자를 눌러도 매번 다른 결과가 나오며, 만들 수 있는 조합이 천문학적이라 독일군은 절대 해독 불가능하다고 확신했습니다.

- **튜링과 봄베** — 영국 블레츨리 파크의 앨런 튜링이 '봄베'라는 계산 장치를 개발해 에니그마를 해독했습니다. 날씨 보고서처럼 예측 가능한 내용('크립')을 단서로 활용했으며, 연합군이 독일군 통신을 실시간 해독하면서 전쟁을 약 2 년 앞당긴 것으로 평가됩니다.

섀넌 — 암호학을 과학으로

1949년 미국 수학자 클로드 섀넌이 발표한 '비밀 시스템의 통신 이론'은 현대 암호학의 수학적 기초를 마련했습니다. 그는 암호 만들기를 직관과 경험의 기술에서 수학적 과학으로 바꾸어 놓았습니다.

- **완전 보안** — 암호문만으로는 원문에 대한 어떤 정보도 얻을 수 없는 상태. 아무리 뛰어난 해독자라도 키 없이는 절대 풀 수 없습니다.
- **일회용 패드** — 완전 보안을 달성하는 암호로, 키의 길이가 원문과 같거나 길고, 진정한 무작위로 생성되며, 각 키는 단 한 번만 사용되어야 합니다. 이론적으로 완벽하지만 긴 키를 안전하게 미리 공유해야 하는 실용적 한계가 있습니다.

> **핵심 요점** 단순 치환 → 빈도 분석으로 붕괴 → 다중 알파벳 → 기계식(에니그마) → 섀넌의 수학적 기초로 이어지는 '암호와 해독의 끝없는 경쟁'이 암호학 발전의 핵심 흐름입니다.

2 장. 대칭키 암호화의 원리와 활용

대칭키 암호화는 송신자와 수신자가 동일한 키를 사용해 데이터를 암호화·복호화하는 방식입니다. 하나의 키가 잠금과 열림 두 역할을 모두 수행하며, 무엇보다 속도가 빨라 대용량 파일이나 실시간 통신에 유용합니다.

왜 같은 키를 써야 하는가

암호화의 목적은 허가받지 않은 사람이 정보를 읽지 못하게 하는 것입니다. 암호화 키와 복호화 키가 다르다면 복호화 키를 모르는 사람도 암호문을 만들 수 있어 보안 허점이 됩니다. 같은 키를 사용함으로써 키를 아는 사람만이 암호화·복호화할 수 있도록 보장합니다.

케르크호프스의 원리

19세기 네덜란드 언어학자 오귀스트 케르크호프스가 제시한 원칙으로, 암호화 알고리즘은 공개되어도 상관없지만 키는 반드시 비밀로 유지되어야 한다는 것입니다. 시스템의 보안은 알고리즘의 비밀성이 아니라 오직 키의 비밀성에만 의존해야 합니다. 알고리즘이 노출되어도 키만 바꾸면 되고, 공개된 알고리즘은 많은 전문가의 검증을 받아 더 안전합니다.

표준 암호 알고리즘의 발전

- **DES (데이터 암호화 표준)** — 1977 년 미국이 채택. IBM 의 루시퍼 알고리즘 기반이며 NSA 가 관여했습니다. 56 비트 키를 사용했으나 컴퓨터 성능 향상으로 1997 년 특수 컴퓨터가 몇 시간 만에 키를 찾아내면서 안전성을 상실했습니다.
- **AES (고급 암호화 표준)** — 2001 년 정식 채택. 공개 공모를 거쳐 벨기에 암호학자들이 개발한 리즈나엘(Rijndael) 알고리즘이 선정되었습니다. 128·192·256 비트 키를 지원하며 현재 세계에서 가장 널리 쓰이는 대칭키 표준입니다.

블록 암호 vs 스트림 암호

- **블록 암호** — 데이터를 일정 크기 블록으로 나눠 암호화(DES 는 64 비트, AES 는 128 비트 블록). 책을 챕터별로 나눠 읽는 것에 비유됩니다.
- **스트림 암호** — 데이터를 비트·바이트 단위로 연속 처리. 키스트림(의사 난수)을 생성해 원본과 XOR 연산으로 암호화하며 실시간·스트리밍에 적합. 대표적으로 RC4 가 있습니다.

블록 암호의 운영 모드

- **ECB 모드** — 각 블록을 독립 암호화. 간단하지만 같은 평문 블록이 항상 같은 암호문이 되어 원본 패턴이 드러나는 치명적 약점이 있습니다.

- **CBC 모드** — 이전 블록의 암호문이 다음 블록에 영향을 주는 체인 방식. 첫 블록에는 초기화 벡터(IV)를 사용해 매번 다른 결과가 나오게 합니다.
- **CFB / OFB 모드** — 블록 암호를 스트림 암호처럼 사용하게 해주는 방식으로 실시간 데이터 전송에 유용합니다.

대칭키의 근본적 한계 — 키 분배 문제

아무리 강력한 알고리즘도 송신자와 수신자가 키를 안전하게 공유하지 못하면 무의미합니다. 100명과 안전하게 통신하려면 100개의 서로 다른 키를 관리해야 하고, 각 키를 안전하게 전달·보관·갱신해야 합니다. 이 문제 때문에 실제 환경에서는 공개키 암호화와 함께 쓰는 하이브리드 방식이 널리 채택됩니다(예: HTTPS는 공개키로 대칭키를 안전하게 교환한 뒤, 빠른 대칭키로 실제 데이터를 전송).

> **핵심 요점** 대칭키는 '빠르지만 키를 나눠 갖기 어렵다'가 핵심. 이 약점을 공개키가 보완합니다.

3 장. 공개키 암호화의 혁신적 발견

1970년대 중반 암호학계에는 수세기 동안 풀리지 않던 근본 문제, 즉 '키 분배 문제'가 있었습니다. 키를 가로챌 수 있는 사람이 있다면 암호화가 무의미해지는데, 컴퓨터가 1000대만 통신해도 약 50만 개의 키가 필요할 만큼 관리해야 할 키 수가 기하급수적으로 늘어났습니다.

디피-헬만 키 교환

1976년 스탠퍼드 대학의 휘트필드 디피와 마틴 헬만이 논문 '뉴 디렉션스 인 크립토그래피'에서 혁명적 아이디어를 제시했습니다. 암호키를 두 개로 나누되 하나는 공개하고 하나는 비밀로 보관하자는 것이었습니다.

- **물감 혼합 비유** — 앨리스와 밥이 공통 색(노란색)을 공개적으로 정하고, 각자 비밀 색(빨강·파랑)을 섞어 주황·초록을 만들어 교환합니다. 여기에 다시 자신의 비밀 색을 섞으면 둘 다 같은 최종 색을 얻습니다. 도청자가 공개된 색을 모두 알아도 비밀 색은 알아낼 수 없습니다.
- **일방향 특성** — 물감이 섞이면 원래 색으로 분리하기 어렵듯, 수학적으로도 특정 문제는 한 방향은 쉽게 계산되지만 역방향은 극도로 어렵습니다.

RSA 알고리즘

디피-헬만은 이론적 가능성만 제시했을 뿐 실용적 구현 방법은 명확하지 않았습니다. 1977년 MIT의 로널드 리베스트, 아디 샤미르, 레오나드 애들먼이 구체적 방법을 개발했고, 이들 이름 첫 글자를 딴 것이 RSA입니다.

- **소인수분해의 어려움** — 두 개의 큰 소수를 곱해 더 큰 수를 만드는 것은 쉽지만, 그 큰 수에서 원래 두 소수를 찾아내는 것은 매우 어렵다는 사실을 이용합니다. 수가 커질수록 분해는 급격히 어려워져, 수백 자리 수는 현재 컴퓨터로도 수천 년이 걸릴 수 있습니다.
- **상업화** — 1982 년 RSA 데이터 시큐리티 회사 설립으로 암호학이 순수 학문에서 상업적 응용으로 전환되었고, 일반 기업도 강력한 암호화를 쓸 수 있게 되었습니다.

공개키와 개인키의 관계 · 두 가지 용도

두 키는 수학적으로 밀접하나 한쪽을 알아도 다른 쪽을 추론할 수 없습니다. 한 키로 잠근 것은 반드시 다른 키로만 열 수 있으며, 이 중 하나는 전 세계에 공개해도 안전합니다.

- **암호화** — 상대가 내 공개키로 암호화해 보내면, 나만 가진 개인키로만 해독할 수 있습니다.
- **디지털 서명** — 내 개인키로 암호화(서명)하면, 다른 사람들이 내 공개키로 해독함으로써 그 문서가 정말 나로부터 왔음을 확인할 수 있습니다.

발전과 새로운 위협

- **타원곡선 암호(ECC)** — RSA 보다 훨씬 작은 키로 같은 수준의 보안을 제공. 처리 능력·저장 공간이 제한된 스마트폰·IoT 기기에 유용합니다.
- **PKI(공개키 기반 구조)** — 공개키 인증서를 발급·관리하는 전체 시스템. 인증기관이 공개키 소유자의 신원과 디지털 서명이 담긴 인증서를 발급해 진위를 확인할 수 있게 합니다.
- **양자 컴퓨터 위협** — 쇼어 알고리즘으로 소인수분해를 효율적으로 풀 수 있어, 수천 년 걸릴 RSA 공격을 몇 시간~며칠 만에 해낼 것으로 예측됩니다.
- **포스트 양자 암호화** — 양자내성 암호. 격자 문제, 다변수 방정식, 해시 기반 서명 등을 기반으로 하며 미국 NIST 등에서 표준화 작업이 진행 중입니다.

> **핵심 요점** 공개키 암호화는 온라인 쇼핑·인터넷 뱅킹·이메일 등 오늘날 디지털 세상의 신뢰 기반을 만든 혁신입니다.

4 장. 해시함수와 디지털 서명의 비밀

해시함수는 임의 크기의 데이터를 고정 크기 값으로 변환하는 수학적 함수로, 도서 분류 번호처럼 데이터를 식별할 뿐 원본 내용을 담지는 않습니다.

해시함수의 두 가지 핵심 특성

- **일방향성** — 해시값을 보고 원본 데이터를 역추적하는 것은 거의 불가능합니다. 다진 고기를 보고 원래 고기 모양을 알 수 없는 것에 비유됩니다.
- **충돌 저항성** — 서로 다른 두 입력이 같은 해시값을 만들어내지 못하게 하는 능력. 좋은 해시함수는 다른 데이터에 대해 다른 해시값을 생성해야 하며, 충돌이 발견되면 보안성이 크게 훼손됩니다.

해시함수의 발전사

- **MD5** — 1990 년대 초 로널드 리베스트가 개발, 128 비트 해시값 생성. 파일 무결성 검증·비밀번호 저장에 널리 쓰였으나, 2004 년 왕샤오윈이 효율적 충돌 방법을 발견하고 2008 년 가짜 인증서 제작까지 공개되며 보안 용도 사용이 권장되지 않게 되었습니다.
- **SHA-1** — 160 비트 해시값 생성으로 MD5 보다 안전했으나, 2005 년경 이론적 공격이 발견되고 2017 년 구글이 실제 충돌을 찾아내면서 안전하지 않음이 증명되었습니다.
- **SHA-2** — 현재 가장 널리 쓰이는 계열(SHA-224/256/384/512). 특히 SHA-256 은 비트코인 등 블록체인에서 사용되며 아직 실용적 공격이 발견되지 않은 안전한 해시함수로 여겨집니다.
- **SHA-3** — 2015 년 표준 승인. SHA-2 와 완전히 다른 구조라 SHA-2 에 문제가 생겨도 대안으로 쓸 수 있습니다.

디지털 서명

디지털 서명은 공개키 암호화를 바탕으로 하여 위조가 거의 불가능하고, 문서마다 고유한 서명이 생성되어 다른 문서에 복사할 수 없습니다.

- **작동 원리** — ① 문서를 해시함수에 통과시켜 해시값 생성 → ② 자신의 개인키로 해시값을 암호화(=디지털 서명) → ③ 받는 사람은 서명자의 공개키로 서명을 복호화해 해시값을 얻고, 받은 문서를 같은 해시함수에 통과시켜 계산한 해시값과 비교 → ④ 두 값이 일치하면 위변조되지 않았고 정당한 서명자가 서명했음이 확인됩니다.
- **부인 방지** — 개인키는 소유자만 갖고 있으므로 '그런 거래를 한 적 없다'는 부인을 방지합니다. 2000 년 미국, 2001 년 한국에서 전자서명법이 시행되어 필기 서명과 동등한 법적 효력을 인정받았습니다.

메시지 인증 코드(MAC)와 HMAC

- **MAC** — 메시지의 무결성과 인증을 동시에 제공. 일반 해시함수는 누구나 계산할 수 있지만 MAC 은 비밀키를 아는 사람만 계산할 수 있습니다. 송·수신자가 비밀키를 공유하고 각자 계산한 MAC 값을 비교해 위변조와 발신자를 확인합니다.
- **HMAC** — 가장 널리 쓰이는 MAC 알고리즘. 기존 해시함수를 활용하며 SHA-256 과 결합한 HMAC-SHA256 형태로 API 인증·메시지 무결성 검증에 쓰입니다.

> **핵심 요점** 해시함수(무결성), 디지털 서명(인증·부인방지), MAC(무결성+인증)은 함께 쓰여 안전한 디지털 세상의 '보이지 않는 방패' 역할을 합니다.

5 장. 키 관리와 인증서의 세계

디지털 열쇠인 암호키는 물리적 열쇠보다 훨씬 복잡하고 정교한 관리가 필요합니다. 은행 시스템의 키가 유출되면 수백만 명의 고객 정보가 위험해지기 때문입니다.

암호키의 생명주기

- **생성** — 충분히 복잡하고 예측 불가능한 키를 만들어야 합니다. 컴퓨터는 진짜 무작위 수를 만들기 어려워 마우스 움직임·키보드 입력 등 인간 행동 패턴, 하드웨어 잡음 등으로 예측 어려운 씨앗값을 만듭니다.
- **분배** — 대칭키에서는 키를 안전하게 전달하는 것이 핵심. 공격자가 키를 가로채면 모든 정보가 위험해지므로 디피-헬만 키 교환이나 공개키 방식을 활용합니다.
- **저장** — 소프트웨어로만 저장하면 해킹 위험이 있어 하드웨어 보안 모듈(HSM)을 사용합니다. HSM 은 물리적으로 분리된 하드웨어 내부에서 키를 생성·저장·연산하며, 물리적 공격 시 내부 키가 자동 삭제되는 자폭 기능도 갖춥니다.
- **사용** — 누가·언제·어떤 목적으로 키를 쓰는지 철저히 기록·관리하고, 사용 빈도·패턴을 모니터링해 비정상 사용을 탐지합니다.
- **갱신** — 자동차 엔진오일 교체처럼 정기적으로 키를 교체. 알고리즘이 보안 요구에 맞지 않거나 유출 가능성이 있으면 즉시 교체합니다.
- **폐기** — 하드디스크 물리 섹터에 남은 데이터까지 완전히 지우고, 때로는 하드디스크 자체를 물리적으로 파괴합니다.

공개키 인증서와 인증기관(CA)

가짜 공개키를 진짜인 것처럼 속이는 문제를 해결하기 위해 등장한 것이 공개키 인증서입니다. 인증서에는 키 소유자의 신원 정보, 공개키, 신뢰할 수 있는 인증기관의 디지털 서명이 담깁니다. 인증기관(Certificate Authority, CA)은 정부처럼 개인·조직의 신원을 확인하고 인증서를 발급합니다. 대표적으로 베리사인은 1995년 설립되어 전자상거래 발전에 기여했습니다.

브라우저의 인증서 검증 과정

- 웹사이트가 제시한 인증서가 신뢰할 수 있는 인증기관에서 발급되었는지 확인(브라우저에 루트 인증기관 목록이 미리 저장).
- 인증서의 도메인이 실제 접속한 도메인과 일치하는지 확인.
- 인증서 유효 기간이 만료되지 않았는지, 폐기되지 않았는지 확인.

- 폐기 목록 확인 또는 OCSP 프로토콜로 실시간 인증서 상태 조회. 개인키가 유출되면 해당 인증서를 즉시 폐기 목록에 등록합니다.

키 에스크로 논쟁

1990년대 미국 정부는 클리퍼 칩을 개발해 모든 통신 장비에 의무 탑재하려 했습니다. 이 칩에는 정부가 복사본을 보관하는 마스터 키가 들어 있어 필요시 암호화된 통신을 해독할 수 있도록 설계되었습니다. 시민단체와 기업은 '정부가 모든 집의 열쇠 복사본을 보관하는 것과 같다'며 강력히 반발했고, 국가안보·범죄 수사(정부)와 개인 프라이버시·표현의 자유(시민사회) 사이의 균형 논쟁은 오늘날까지 이어지고 있습니다.

> **핵심 요점** 키 관리와 인증서 시스템은 온라인 쇼핑·뱅킹·이메일을 가능하게 하는, 현대 디지털 사회의 '보이지 않는 신뢰 기반'입니다.

6 장. 네트워크 보안의 기초와 실제

물리적 편지와 달리 네트워크상의 데이터는 중간 과정에서 누군가가 엿볼 수 있다는 점이 중요한 차이입니다.

OSI 7 계층과 계층별 위협

- **물리 계층** — 전기·광신호로 데이터 전송. 케이블에 직접 접근한 도청 위협.
- **데이터 링크 계층** — 같은 세그먼트 내 MAC 주소 조작 공격(문패를 바꿔 다는 것에 비유).
- **네트워크 계층** — IP 주소 스푸핑 — 자신의 실제 IP 를 숨기고 다른 IP 인 것처럼 위장.
- **전송 계층** — TCP·UDP 대상 공격. 특히 SYN 플러딩은 연결 요청만 계속 보내 서버 대기 공간을 채워 정상 사용자 접속을 막습니다.
- **세션 계층** — 통신 세션 하이재킹(전화 통화에 끼어들어 가로채는 것에 비유).
- **표현/응용 계층** — 데이터 압축·암호화 과정의 취약점, 웹 브라우저·이메일 클라이언트 등 사용자 프로그램이 공격 대상.

주요 공격 기법

- **IP 스푸핑** — 발신 주소를 거짓으로 위장. 1995 년 케빈 미트닉 사건이 유명하며, 방어를 위해 송신자 인증과 패킷 필터링이 중요합니다.
- **네트워크 스니핑** — 네트워크상 데이터 패킷을 가로채 분석. 와이어샤크가 대표 도구로, 합법적 관리에도 쓰이지만 암호화되지 않은 비밀번호·개인정보가 노출될 수 있습니다.
- **중간자 공격(MITM)** — 송·수신자 사이에 몰래 위치해 통신을 가로채고 조작. 양쪽 모두 직접 통신한다고 믿는 것이 가장 위험한 점입니다. 공개 와이파이에서 '무료 인터넷'이라는 가짜 핫스팟(이블 트윈)을 만들어 트래픽을 가로채는 사례가 대표적입니다.

방어 기술

- **HTTPS** — HTTP 에 SSL/TLS 암호화를 추가한 프로토콜. 공개키로 초기 연결을 설정한 뒤 대칭키로 실제 데이터를 암호화합니다.
- **VPN** — 공용 네트워크에 사설 네트워크를 구축. 원격 접속 VPN(개별 사용자↔회사망)과 사이트 간 VPN(지역이 다른 사무실 연결)이 있으며, 코로나 19 재택근무로 사용이 급증했습니다.
- **NAC(네트워크 접근 제어)** — 접속 장치의 하드웨어·OS 버전·보안 SW 상태를 검사해 보안 정책에 부합하는 장치만 접속을 허용합니다.

심층 방어 전략

현대 네트워크 보안은 방화벽 하나로 해결할 수 없어, 성을 방어할 때 성벽·해자·망루·성문을 두듯 방화벽·침입탐지시스템(IDS)·VPN·접근제어·암호화 등 여러 방어선을 조합합니다. 하나가 뚫려도 다른 방어선에서 공격을 막을 수 있게 하는 것이 핵심입니다.

> **핵심 요점** 계층마다 위협이 다르므로 계층별 대응과 다층 방어(심층 방어)가 네트워크 보안의 기본 원칙입니다.

7 장. 웹 보안과 사이버 공격들

웹 환경의 보안 위협은 일상과 매우 밀접하며 그 기법은 날로 정교해지고 있습니다(2008년 한국 대형 포털 1,800만 명 개인정보 유출 사례).

SQL 인젝션

SQL 명령문에 악의적 코드를 삽입하는 공격입니다. 로그인 창에 아이디 대신 «admin'; DROP TABLE users; --» 같은 코드를 입력하면, 데이터베이스가 이를 정상 명령으로 인식해 테이블 전체가 삭제될 수 있습니다. 정상적인 입력 경로로 이루어져 방화벽·침입탐지시스템으로도 탐지하기 어렵습니다.

- **실제 사례** — 2011 년 소니 플레이스테이션 네트워크 해킹으로 7,700 만 명의 개인정보가 유출, 서비스가 23 일간 중단되고 피해는 약 1 억 7,100 만 달러, 주가는 10% 이상 하락했습니다.

XSS (크로스 사이트 스크립팅)

게시판·댓글 기능을 악용해 악성 스크립트를 삽입하는 공격입니다. «<script>alert('해킹당했습니다!')</script>» 같은 코드를 게시글에 넣으면 이를 조회하는 모든 사용자 브라우저에서 실행되어, 쿠키 탈취·세션 가로채기·개인정보 전송이 가능합니다.

- **저장형 XSS** — 악성 스크립트가 웹서버 DB 에 영구 저장되어 게시글을 보는 모든 사용자에게 영향(게시판·댓글).
- **반사형 XSS** — URL 파라미터·폼 입력으로 전달되어 즉시 실행(검색·오류 메시지).
- **DOM 기반 XSS** — 클라이언트 측 자바스크립트가 DOM 을 조작하는 과정에서 발생.

CSRF (크로스 사이트 요청 위조)

사용자가 이미 인증된 상태를 악용하는 공격입니다. 은행에 로그인한 상태에서 악성 코드가 숨은 사이트를 방문하면, 사용자 모르게 송금 요청이 전송될 수 있습니다. 브라우저가 같은 도메인 요청에 쿠키를 자동 전송하는 특성을 이용하며, 이미지·폼 태그로 은밀하게 요청을 생성합니다. 2008년 유명 동영상 공유 사이트에서 사용자 모르게 친구 추가 요청을 보내게 만든 사건이 있었습니다.

웹 애플리케이션 방화벽(WAF)

기존 방화벽이 IP·포트 기준으로 차단하는 것과 달리, WAF는 HTTP/HTTPS 트래픽 내용을 분석해 악의적 요청을 탐지·차단합니다.

- **시그니처 기반 탐지** — 미리 정의된 공격 패턴과 일치하는 요청 차단(예: 'UNION SELECT', 'DROP TABLE', '<script>', 'javascript:').

- **행위 기반 탐지** — 정상 트래픽과 다른 비정상 요청을 탐지. 머신러닝을 활용해 새로운 형태의 공격도 효과적으로 차단합니다.

> **핵심 요점** SQL 인젝션·XSS·CSRF는 모두 '신뢰받는 입력·인증 상태'를 악용합니다. WAF는 이를 걸러내는 검색대 역할을 합니다.

8 장. 무선 네트워크 보안의 이해

무선 통신은 전파를 매체로 사용해, 적절한 수신 장비만 있으면 주변 누구나 신호를 포착할 수 있어 태생적으로 보안에 더 큰 주의가 필요합니다.

무선의 3 가지 근본 위험

- **도청** — 신호가 물리적 경계 없이 퍼져 안테나만 있으면 멀리서도 트래픽을 엿들을 수 있습니다.
- **불법 접근** — 유선과 달리 신호 도달 범위 내 어디서든 접속을 시도할 수 있습니다.
- **중간자 공격** — 가짜 무선 공유기를 설치해 진짜인 줄 알고 접속하도록 유도한 뒤 통신을 가로챕니다.

무선 보안 표준의 진화

- **WEP** — 유선과 동등한 프라이버시를 목표로 한 첫 표준. RC4 알고리즘과 24 비트 초기화 벡터(IV)를 사용했으나, IV 가 너무 짧아 금세 중복되고 평문으로 전송되는 치명적 결함이 있었습니다. 2001 년 플러겔·만틴·샤미르가 취약점을 공개, 에어크랙 같은 도구로 몇 시간 내 해독이 가능해졌습니다.
- **WPA** — TKIP 프로토콜로 패킷마다 다른 키를 쓰는 동적 키 생성을 채택하고 키 길이를 128 비트로 확장, 메시지 무결성 검사도 추가했습니다. 다만 여전히 RC4 기반이라 근본적 한계가 있었고 2008 년 WPS 핀 번호 무차별 대입 등 취약점이 드러났습니다.
- **WPA2** — AES 암호화 알고리즘 사용으로 대폭 개선. 256 비트 키는 현재 기술로 수십억 년이 걸릴 만큼 강력합니다. CCMP 프로토콜로 기밀성과 무결성을 한 과정에서 처리했습니다. 그러나 2017 년 KRACK 공격(4-way 핸드셰이크에서 키를 두 번 설치하도록 유도해 카운터 값 재사용)으로 취약점이 드러났고, 소프트웨어 업데이트로 해결되었습니다.

블루투스 보안 위협

- **블루재킹** — 원치 않는 메시지·이미지를 전송(주로 장난·광고, 때로 악성 파일 전파).
- **블루스나핑** — 블루투스 연결로 상대 기기의 연락처·메시지·통화 내역까지 무단 접근하는 심각한 공격.
- **블루본** — 페어링 과정의 취약점을 이용해 두 기기 사이 통신을 가로챕니다. 가시성 설정 비활성화·정기 보안 업데이트가 방어책입니다.

공개 와이파이 주의사항과 5G

- 공개 와이파이는 미암호화·약한 보안이 많고 이블 트윈 공격이 빈번 → VPN 사용, 중요한 금융 거래 회피, HTTPS 사이트만 이용.
- 5G 는 상호 인증 강화, 암호화 견고화, 네트워크 슬라이싱으로 서비스별 독립 보안 정책 적용이 가능해졌으나 복잡성으로 예상 못한 취약점 가능성도 있어 지속적 모니터링이 필요합니다.

> **핵심 요점** WEP → WPA → WPA2로의 진화는 '완벽해 보이는 보안도 언제든 새 공격에 취약할 수 있다'는 교훈을 남겼습니다.

9 장. 운영체제 보안과 시스템 보호

운영체제는 컴퓨터의 모든 자원을 관리하는 핵심 소프트웨어로, 그 보안 허점이 2017년 워너크라이 사태처럼 대규모 피해의 시작점이 되기도 합니다.

다단계 보안 모델

- **벨-라파듈라 모델** — 1970 년대 미군이 기밀문서 관리를 위해 개발. '위로 읽기 금지, 아래로 쓰기 금지' 두 원칙으로 정보가 높은 등급에서 낮은 등급으로 새어나가는 것을 방지합니다.
- **UAC(사용자 계정 제어)** — «이 앱이 디바이스를 변경할 수 있도록 허용하시겠습니까?» 팝업이 대표적. 중요한 결정 전 한 번 더 확인하게 합니다.
- **최소 권한 원칙** — 사용자·프로그램에 작업 수행에 필요한 최소한의 권한만 부여. 시스템 파일 수정·프로그램 설치 등에는 관리자 권한이 필요합니다.

주요 공격 — 버퍼 오버플로우

프로그램이 예상보다 많은 데이터를 받을 때 발생합니다. 10글자까지 저장하는 공간에 100글자를 입력하면 넘치는 데이터가 다른 중요 정보를 덮어씁니다. 특히 스택(나중에 들어온 정보가 먼저 나가는 구조)에서 함수의 반환 주소를 조작하면 공격자가 원하는 악성 코드를 실행하게 만들 수 있습니다. 1988년 모리스 웜이 이 방식으로 당시 연결 컴퓨터 6만 대 중 10%를 감염시켰습니다.

- **방어 기법** — 스택 가드(버퍼와 반환 주소 사이 특별한 값을 넣어 변경 여부 확인), ASLR(주소 공간 배치 무작위화로 공격자가 정확한 주소를 예측하기 어렵게 함).

루트킷

투명인간처럼 시스템에 숨어 활동하는 악성 프로그램으로, 부트로더·커널 등 가장 깊은 곳에 침투해 자신의 존재를 철저히 감춥니다. 보안 프로그램이 파일·프로세스 목록을 요청하면 자신과 관련된 항목을 제외시켜 탐지가 어렵습니다. 2005년 소니 BMG 음악 CD에 숨겨진 루트킷 사건이 대표적입니다.

가상화와 종합 대응

- **가상화** — 하나의 물리 컴퓨터 위에 독립된 가상 머신을 만들어, 한 가상 머신이 감염돼도 다른 곳에 영향을 주지 않습니다. 하이퍼바이저가 각 가상 머신을 관리·격리합니다. 위험한 파일 실행·의심 사이트 방문 시 유용합니다.
- **시스템 모니터링·로그 분석** — 평소 오전 9 시에만 접속하던 사용자가 갑자기 새벽 3 시에 로그인하면 계정 탈취를 의심할 수 있습니다.

- **정기 보안 업데이트** — 2017 년 워너크라이는 마이크로소프트가 이미 패치를 제공한 취약점을 노렸고, 패치 미적용 시스템만 피해를 입었습니다.

> **핵심 요점** 운영체제 보안은 결국 다층 방어입니다. 계정 제어·방화벽·안티바이러스·침입탐지가 협력해 한 곳이 뚫려도 다른 곳에서 막습니다.

10 장. 데이터베이스 보안과 악성코드의 진화

2013년 미국 대형 소매업체 타겟에서 4천만 명의 신용카드 정보가 유출되는 등, 중요한 정보가 한곳에 모인 데이터베이스는 '양날의 검'입니다.

데이터베이스 보안의 방어선

- **접근 제어** — 각 사용자에게 역할·책임에 맞는 권한 부여(고객 서비스 직원은 주문 내역만, 결제 담당자는 결제 정보만). 최소 권한 원칙을 적용합니다.
- **암호화** — 전체 DB 암호화(성능 저하), 컬럼 단위 암호화(주민번호·카드번호 등 중요 정보만 선택 암호화), 투명한 데이터 암호화(TDE, 자동 처리)가 있습니다. 주민번호 «850315-1234567»을 «X7K9P2M5Q8N3B1» 같은 무의미한 문자열로 저장하는 식입니다.
- **감사 로그** — 누가·언제·어떤 데이터에 접근해 어떤 작업을 했는지 기록. 평소 업무 시간에만 접근하던 직원이 새벽 3 시에 대량 조회하면 경고가 발생합니다. AI·머신러닝으로 평소 패턴을 학습해 이상 행위를 정교하게 탐지합니다.

프라이버시 보호 기술

- **비식별화** — 개인 식별 정보를 제거·변형(나이 '25 세'→'20 대', '서울시 강남구 역삼동'→'서울시').
- **가명화** — 실제 이름 대신 무작위 가명 사용('김철수'→'A001'). 단, 다른 정보와 조합하면 재식별 가능성이 있어 단독으로는 부족합니다.
- **k-익명성** — 데이터셋에서 동일 속성을 가진 사람이 최소 k 명 이상 존재하도록 만들어(예: k=3), 특정 개인을 식별할 수 없게 합니다.
- **차분 프라이버시** — 쿼리 결과에 수학적으로 계산된 노이즈를 추가해 특정 개인 포함 여부를 알 수 없게 하는 고급 기법(애플·구글 등 활용).
- **DLP(데이터 유출 방지)** — 공항 보안 검색대처럼 중요 데이터가 조직 밖으로 무단 반출되는 것을 감시·차단. 2013 년 스노든 사건처럼 정당한 권한을 가진 내부자의 유출에 대응합니다.

악성코드의 진화

- **크리퍼 (1971)** — ARPANET 에서 «I'M THE CREEPER, CATCH ME IF YOU CAN!» 메시지를 띄운 최초의 컴퓨터 바이러스. 이를 제거한 리퍼가 최초의 안티바이러스 원형이었습니다.
- **브레인 (1986)** — 파키스탄에서 만들어진 최초의 PC 바이러스. 인터넷 이전이라 플로피 디스크·CD-ROM 등 물리 매체로 전파되었습니다.
- **웜** — 바이러스와 달리 다른 프로그램에 기생하지 않고 독립적으로 존재하며 네트워크를 통해 스스로 복제·전파. 시스템 자원과 네트워크 대역폭을 잠식합니다.

- **트로이목마** — 유용한 프로그램으로 위장해 사용자가 자발적으로 설치하도록 유도. 스스로 복제하진 않지만 백도어 설치·개인정보 탈취 기능을 숨깁니다.
- **스파이웨어·키로거** — 웹 브라우징 습관·키보드 입력·개인정보를 수집해 외부로 전송. 키로거는 모든 키 입력을 기록해 비밀번호·카드번호를 탈취합니다.
- **애드웨어·스케어웨어** — 애드웨어는 원치 않는 광고를 무작정 띄우고, 스케어웨어는 가짜 안티바이러스로 위장해 존재하지 않는 바이러스를 경고하며 유료 구매를 유도합니다.
- **랜섬웨어 (워너크라이, 2017)** — 2017 년 5 월 12 일 전 세계 150 여 개국 30 만 대 이상의 컴퓨터를 감염시켜 병원·철도·정부기관을 마비시킨 전례 없는 글로벌 사이버 대란을 일으켰습니다.

랜섬웨어와 워너크라이 사태의 의미

랜섬웨어는 사용자의 파일을 암호화해 읽을 수 없게 만든 뒤, 복구 대가로 비트코인 몸값을 요구하는 악성코드입니다. 납치범이 인질을 잡고 몸값을 요구하듯, 사용자의 소중한 데이터를 인질로 삼습니다. 워너크라이는 미국 국가안보청(NSA)이 발견한 윈도우 운영체제 취약점을 악용했는데, 이 취약점 정보가 해킹으로 유출되면서 전 세계적 피해로 이어졌습니다.

- **실제 피해** — 영국 국민보건서비스(NHS) 병원이 마비되어 수술이 취소되고, 독일 철도 시스템이 중단되었으며, 러시아 내무부·통신회사, 한국의 영화관·주유소 등도 업무를 멈춰야 했습니다.
- **국가 안보 문제로의 전환** — 사이버 공격이 개인·기업을 넘어 병원·교통·통신·금융 등 사회 기반시설을 마비시킬 수 있음을 드러냈고, 이후 각국 정부는 사이버 보안을 국가 안보의 핵심 영역으로 인식하고 대응 체계를 강화하기 시작했습니다.

고도화된 현대의 공격 유형

- **제로데이 공격** — 소프트웨어 개발자조차 모르는 새로운 취약점(제로데이)을 악용하는 공격. 아직 발견되지 않은 숨겨진 문으로 침입하는 것과 같아 기존 보안 솔루션으로는 탐지·방어가 매우 어렵습니다. 취약점이 발견·악용되는 시점과 보안 전문가가 패치를 개발하는 시점 사이의 시간차를 노립니다.
- **APT 공격 (지능형 지속 위협)** — Advanced Persistent Threat 의 약자로, 고도로 숙련된 공격자가 특정 목표를 겨냥해 장기간 은밀하게 수행하는 공격. 적진에 잠입한 스파이처럼 한 번 침입하면 발각되지 않도록 조심스럽게 활동하며 지속적으로 정보를 수집하고 더 깊숙이 침투합니다.
- **APT 의 특징** — 목표의 명확성과 공격의 지속성. 무차별 공격하는 일반 악성코드와 달리 특정 기업·정부기관을 목표로 원하는 정보를 얻을 때까지 수개월~수년간 지속합니다. 직원 대상

스피어 피싱 이메일, 자주 방문하는 사이트에 악성코드를 심는 워터링 홀 공격 등으로 초기 침입을 시도합니다.

안티바이러스 탐지 기술의 발전

- **시그니처 기반 탐지** — 알려진 악성코드의 고유 패턴을 데이터베이스에 저장해 두고 파일을 검사할 때 일치 여부를 확인. 지문으로 범인을 식별하는 것과 같지만, 이미 발견된 악성코드만 탐지할 수 있는 한계가 있습니다.
- **휴리스틱 분석** — 파일의 행동 패턴을 분석해 악성코드 가능성을 판단. 의사가 증상을 종합해 질병을 진단하듯, 시스템 파일 무단 수정·의심스러운 데이터 전송·레지스트리 변경 등의 행위를 감지하면 악성코드로 의심합니다.
- **머신러닝 기반 탐지** — 수많은 정상·악성 파일 데이터를 학습해 새로운 파일이 악성일 확률을 계산. 경험 많은 의사가 다양한 사례로 새 환자를 진단하듯, 학습된 패턴으로 이전에 본 적 없는 악성코드도 효과적으로 탐지합니다.
- **행동 기반 탐지 (샌드박스)** — 가상환경인 샌드박스에서 의심 파일을 실제로 실행시켜 그 행동을 실시간 관찰·분석. 새 약물의 부작용을 통제된 실험실에서 테스트하는 것과 유사합니다.
- **클라우드 기반 보안** — 개별 컴퓨터의 제한된 자원 대신 클라우드 서버의 강력한 컴퓨팅 파워로 정밀 분석을 수행. 전 세계 보안 정보를 실시간 공유해 새 악성코드가 발견되면 즉시 모든 사용자를 보호합니다.

맺음말 — 끝없는 경쟁과 사용자의 역할

악성코드와 보안 기술의 경쟁은 앞으로도 계속됩니다. 공격자가 정교한 기법을 개발하면 보안 전문가도 새로운 대응 기술을 개발하는 끝없는 경쟁 속에서, 사용자는 최신 보안 소프트웨어 유지, 의심스러운 이메일·웹사이트 주의, 정기적 백업 등 기본 보안 수칙을 지켜야 합니다. 결국 기술적 발전과 함께 사용자의 보안 의식 향상이 악성코드로부터 우리를 지키는 가장 확실한 방법입니다.

> **핵심 요점** 데이터베이스 보안의 기본 원칙은 '적절한 사람이, 적절한 시간에, 적절한 정보에만 접근하고, 모든 과정을 투명하게 기록·감시하는 것'입니다. 악성코드와 보안 기술의 대결은 크리퍼 이후 지금까지 계속되고 있으며, 기술뿐 아니라 사용자의 보안 의식이 최후의 방어선입니다.